Harry Mendez 24089

Problema 1 Calculo Lambda

a) Reduccion-β de la operacion logica NOT

Codificacion de Church:

TRUE = λx.λy.x

FALSE = λx.λy.y — el booleano escoge

NOT = λp. p FALSE TRUE — uno de sus dos args

NOT TRUE:

(λp. p FALSE TRUE) TRUE

→ TRUE FALSE TRUE — p := TRUE

→ (λx.λy.x) FALSE TRUE

→ (λy. FALSE) TRUE — x := FALSE

→ FALSE

NOT FALSE:

(λp. p FALSE TRUE) FALSE

→ FALSE FALSE TRUE — p := FALSE

→ (λx.λy.y) FALSE TRUE

→ (λy. y) TRUE — x := FALSE

→ TRUE

En cada paso se aplica la funcion a su argumento y se sustituye la variable ligada por el: eso es la reduccion-β.

b) Recursion y ciclos

En calculo lambda las funciones no tienen nombre, asi que una funcion no se puede llamar a si misma. La recursion sale de un combinador de punto fijo, el combinador Y:

Y = λf. (λx. f (x x)) (λx. f (x x))

Propiedad de punto fijo: $Y\,g = g\,(Y\,g)$

$Y\,g = (\lambda f.(\lambda x.\ f\ (x\ x))(\lambda x.\ f\ (x\ x)))\ g$

$\rightarrow (\lambda x.\ g\ (x\ x))\ (\lambda x.\ g\ (x\ x))$

$\rightarrow g\ ((\lambda x.\ g\ (x\ x))\ (\lambda x.\ g\ (x\ x)))$

$= g\ (Y\ g)$ lo de adentro es Y g otra vez

Y se vuelve a expandir solo, y esa expansion infinita es la que hace las veces de recursion.

Ejemplo: factorial

$F = \lambda f.\lambda n.$ IF (ISZERO n) 1 (MULT n (f (PRED n)))

FACT = Y F

FACT 3 → MULT 3 (FACT 2) → MULT 3 (MULT 2 (FACT 1)) → 6

Ciclos:

No hay while ni for, porque no hay estado que mutar ni saltos. Un ciclo se escribe como recursion de cola:

WHILE = Y ($\lambda f.\lambda s.$ IF (cond s) (f (body s)) s)

s es el estado. En vez de modificarlo se pasa un estado nuevo en cada llamada, y para cuando cond s da FALSE: ahi devuelve s.

---

c) Cuando es prudente usarlo y cuando no

Si conviene:

- Transformaciones de datos sin estado (map, filter, reduce) y calculos puros en general.
- Concurrencia y paralelismo: si nada comparte estado no hay condiciones de carrera ni candados.
- Codigo que se tiene que probar o verificar: una funcion pura siempre da el mismo resultado.

Ejemplo: procesar una lista de transacciones para sacar totales por cliente. cada paso es una funcion pura, se puede probar aislada y repartir en varios hilos.

No conviene:

- Cuando hay estado mutable pesado o E/S constante.
- Cuando el rendimiento y la memoria son criticos:
  copiar estructuras inmutables en cada paso gasta
  memoria y recoleccion de basura.
- Cuando el problema es secuencial y lleno de efectos
  (drivers, sistemas embebidos, hardware).

Ejemplo. el motor de un videojuego que actualiza miles de posiciones 60 veces por segundo. Copiar los arreglos en cada frame es carisimo; ahi conviene mutar el arreglo en el lugar.

En resumen: funcional para calculo y transformacion de datos, imperativo para estado y hardware. En la practica casi siempre conviene mezclar los dos.